Leetcode 686 Repeated string match

Bruteforce

```python
class Solution:
    def repeatedStringMatch(self, a: str, b: str) -> int:
        temp = a
        count = 1

        while len(temp) < len(b):
            temp += a
            count += 1

        if b in temp:
            return count

        temp += a
        if b in temp:
            return count + 1

        return -1
```

Code Logic

```
temp = a
count = 1
while len(temp) < len(b):
    temp += a
    count += 1
```

Step 1

temp = "abcd"

count = 1

len(temp) = 4

len(b) = 8

$4 \leq 8 \rightarrow$ True

temp = "abcd abcd"

count = 2

Step -2

len(temp) = 8

len(b) = 8

$8 < 8 \rightarrow$ False

Step 3 → Check Substring

temp = "abcdabcd"

b = "cdabcdab"

Check

"cdab cdab" in "abcd abcd" → False

Step 4 -> Add one extra Repeat

temp += a

temp = "abcd abcd abcd
count = 3

" cda bcdab " in "abcdabcdabcd"

yes, abcd abcd abcd
cda bcd ab

Return 3

Time Complexity -> O(n * m)
Space Complexity -> O(n + m)

Optimal

```python
class Solution:
    def repeatedStringMatch(self, a: str, b: str) -> int:
        count= -(-len(b) // len(a))
        if b in a * count:
            return count

        if b in a * (count +1):
            return count +1


        return -1
```

Code Logic

count = ceil (len(b) / len(a))

compute

len(b) = 8
len(a) = 4

count = ceil (8/4)
count = 2

Step 1 -> check a * count
a * 2

abcd abcd

Check

"cdabcdab" in "abcd abcd"

Result:

False

Step 2:- Check a * (count + 1)

a * 3

"abcdabcdabcd"

Check,

"cdabcdcdab" in "abcdabcdabcd"

Yes,

abcd abcd abcd

cd abcd ab

Return 3

Time complexity → O(n+m) average

Space complexity → O(n)